Construindo uma Pokédex com HTML, CSS e JavaScript

Disciplina: Desenvolvimento Web

Atividade prática individual ou em duplas

API utilizada: PokéAPI

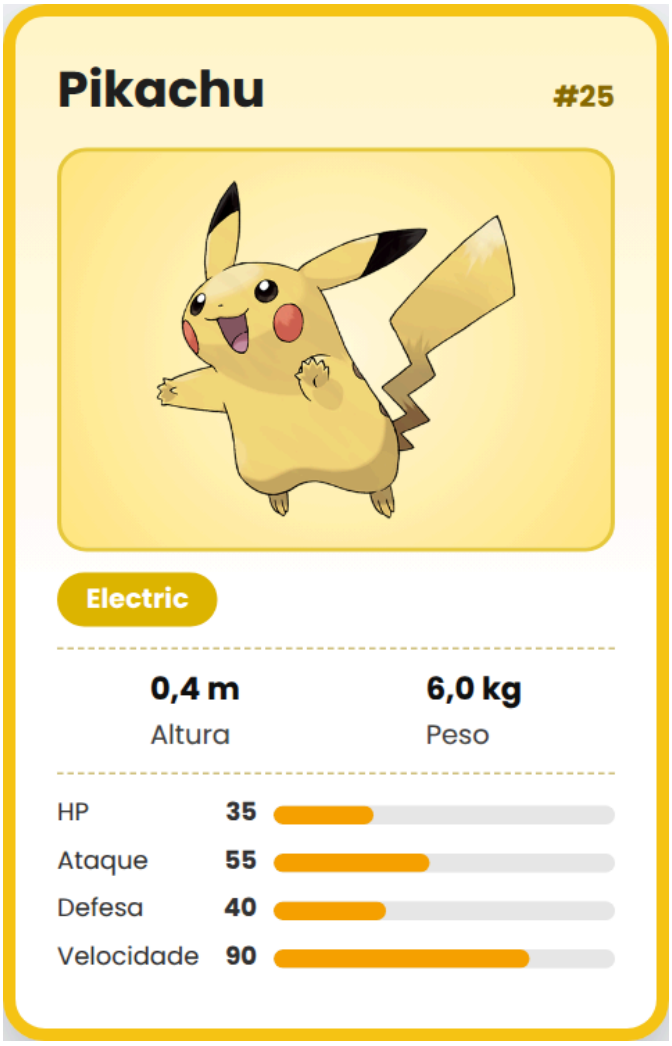
1. Contexto

Você foi contratado(a) para criar a **Pokédex** de um novo treinador. A ideia é simples: o treinador digita o **nome de um Pokémon**, aperta um botão e, na tela, aparece a **cartinha** dele, com imagem, tipos e atributos.

Os dados não ficam no seu código. Eles vêm de uma **API pública e gratuita**, a PokéAPI, que você vai consumir[1] com JavaScript.

A cartinha reúne nome, número, imagem, tipo, altura, peso e barras com os atributos principais.

[1] Consumir uma API significa utilizar os serviços ou dados disponibilizados por ela.



Objetivos de aprendizagem

Ao final da atividade, você deverá ser capaz de:

- Estruturar uma página com HTML semântico (formulário, campo de texto, botão, área de resultado).
- Estilizar componentes com CSS (layout com Flexbox, cores, bordas, sombras, estados de hover).
- Manipular o DOM com JavaScript (selecionar elementos, ouvir eventos, criar e alterar conteúdo).
- Consumir uma API REST com `fetch` e `async/await`.
- Ler e navegar em um objeto JSON.

- Tratar erros (Pokémon inexistente, falha de rede, campo vazio).

3. Conhecendo a PokéAPI

A PokéAPI não exige cadastro nem chave de acesso. Para buscar um Pokémon, basta fazer uma requisição **GET** para:

```
None
https://pokeapi.co/api/v2/pokemon/{nome-ou-id}
```

Exemplos:

- `https://pokeapi.co/api/v2/pokemon/pikachu`
- `https://pokeapi.co/api/v2/pokemon/25`

Atenção: o nome deve estar em **minúsculas** e sem espaços extras. `Pikachu` e `pikachu` podem dar erro. Seu código precisa tratar isso.

3.1 Explore antes de programar

Abra o link do Pikachu no navegador e observe o JSON retornado. Ele é grande, mas você só precisa de alguns campos:

O que mostrar	Onde está no JSON
Nome	`name`
Número na Pokédex	`id`
Imagem	`sprites.other["official-artwork"].front_default`
Tipos	`types[].type.name` (é uma lista)
Altura	`height` (em decímetros: divida por 10 para obter metros)
Peso	`weight` (em hectogramas: divida por 10 para obter kg)
Habilidades	`abilities[].ability.name` (é uma lista)

Atributos (HP, ataque, defesa...)	`stats[].base_stat` e `stats[].stat.name` (é uma lista)

Tarefa 0 (aquecimento): no console do navegador, rode o código abaixo e anote o que aparece.

```javascript
fetch("https://pokeapi.co/api/v2/pokemon/pikachu")
  .then(resposta => resposta.json())
  .then(dados => console.log(dados));
```

4. O que deve ser entregue

Uma página web com os arquivos separados:

```
pokedex/
├── index.html
├── style.css
└── script.js
```

Requisitos obrigatórios

1. Um **campo de texto** para digitar o nome do Pokémon e um **botão "Buscar"**.
2. A busca também deve funcionar ao apertar a tecla **Enter**.
3. Ao buscar, aparecer a **cartinha** contendo no mínimo:
    - imagem do Pokémon;
    - nome e número (ex.: `#25 Pikachu`);

    - tipos;
    - altura e peso;
    - pelo menos 4 atributos (HP, ataque, defesa, velocidade).
4. Se o Pokémon **não existir**, mostrar uma mensagem amigável (e não deixar a página quebrada).
5. Se o campo estiver **vazio**, avisar o usuário sem fazer a requisição.
6. Uma nova busca **substitui** a cartinha anterior.
7. Código organizado em funções, com nomes claros e comentários onde necessário.

5. Roteiro passo a passo

Etapa 1: Estrutura HTML

Crie o `index.html` com:

- um `<header>` com o título da Pokédex;
- um `<form>` contendo `<input>` e `<button>`;
- uma `<section id="resultado">` vazia, onde a cartinha será inserida;
- a ligação com `style.css` e `script.js`.

Pergunta para pensar: por que usar um `<form>` ajuda a fazer o Enter funcionar sem código extra?

Etapa 2: Estilo da cartinha

Antes de ligar a API, crie uma cartinha **fixa** no HTML (com dados fictícios) e estilize até ficar do jeito que você quer. Só depois transforme em algo dinâmico.

Sugestões de CSS:

- largura fixa (ex.: 300px) e `border-radius`;
- `box-shadow` para dar profundidade;
- imagem centralizada e responsiva (`max-width: 100%`);
- tipos como pequenas "etiquetas" (badges) coloridas;
- barras de atributos usando `<div>` com largura proporcional ao valor;
- efeito `:hover` na cartinha (por exemplo, `transform: translateY(-4px)`).

Etapa 3: Conectando com a API

Crie uma função assíncrona que recebe o nome e devolve os dados:

```javascript
function buscarPokemon(nome) {
   return fetch(`https://pokeapi.co/api/v2/pokemon/${nome}`)
       .then(resposta => {
           if (!resposta.ok) {
               throw new Error("Pokémon não encontrado");
           }

           return resposta.json();
       });
}
```

Teste chamando a função no console e inspecionando o resultado.

Etapa 4: Montando a cartinha com os dados

Crie uma função `mostrarCarta(pokemon)` que recebe o objeto retornado pela API e monta o HTML da carta dentro de `#resultado`.

Pontos de atenção:

- `types` e `stats` são **listas**. Use `map()` ou um laço `for...of` para percorrê-las.
- Converta altura e peso para unidades legíveis.
- Deixe a primeira letra do nome em maiúscula.

Etapa 5: Eventos e validações

Ligue tudo no evento de envio do formulário:

- impedir o recarregamento da página (`event.preventDefault()`);
- limpar o texto com `.trim().toLowerCase()`;
- se estiver vazio, avisar o usuário;
- chamar `buscarPokemon` dentro de um `try...catch`;
- em caso de erro, exibir a mensagem na tela.

Etapa 6: Acabamento

- Mostrar "Carregando..." enquanto a requisição acontece.
- Colocar o foco de volta no campo de texto após a busca.
- Testar com pelo menos 5 Pokémon diferentes (incluindo um com dois tipos, como `charizard`) e com entradas inválidas (`xyz`, campo vazio, nome com letras maiúsculas).

6. Desafios extras (opcionais)

- **Cor por tipo:** a cartinha muda de cor conforme o tipo principal (fogo = vermelho, água = azul, planta = verde...).
- **Barras de atributos:** exibir os stats como barras de progresso proporcionais.

- **Botão aleatório:** buscar um Pokémon com número aleatório entre 1 e 1025.
- **Anterior e próximo:** botões para navegar pelo número da Pokédex.
- **Imagem de costas:** alternar entre `front_default` e `back_default` ao clicar na imagem.
- **Histórico:** listar as últimas buscas e permitir clicar para ver de novo.
- **Modo shiny:** botão para alternar para a versão `front_shiny`.
- **Responsividade:** a página funcionar bem em celular.

7. Dicas e erros comuns

- Se aparecer `undefined`, confira o caminho no JSON. Use `console.log(pokemon)` e navegue pelo objeto.
- `await` só funciona dentro de funções `async`.
- Nome com letra maiúscula ou espaço sobrando gera erro 404. Use `.trim().toLowerCase()`.
- A imagem `official-artwork` pode demorar um pouco para carregar. Isso é normal.
- Abra o console (F12) e acompanhe as mensagens de erro: elas ajudam muito.
- Evite copiar código pronto sem entender. Explique cada função com suas palavras.

Entrega

- Compactar a pasta `pokedex/` (ou enviar o link do repositório / GitHub Pages, se a turma já usar).
- Incluir no HTML, em comentário, o nome do(a) aluno(a) ou da dupla.
- Responder em um pequeno texto (3 a 5 linhas) no final do `README.md` ou em comentário: *qual foi a maior dificuldade e como você resolveu?*